

VELA

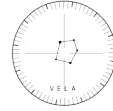
PRIVATE COMMAND INFRASTRUCTURE

Remote Facilitator Guide

Your complete operator playbook for every VELA client installation.
Tailscale SSH access, what to watch for, and how to fix every problem.

Your judgment. Our infrastructure.

Internal Use Only · Greg Shindler · March 2026



This guide is for your eyes only. It contains diagnostic commands, troubleshooting procedures, and client-facing scripts. Do not share with clients.

01 Before the Install Call

Run `vela_deploy.sh` first. Do this the day before.

1 Generate the client's license key

Open Terminal on your Mac Mini.

RUN `bash ~/Downloads/vela-scripts\ 2/vela_deploy.sh`

Fill in the prompts: client name, company, email, tier (Enter for command), pilot or paying.

2 Add the key to the license server

Copy the SQL the script prints. SSH into your droplet.

RUN `ssh root@165.22.36.184`

RUN `sqlite3 /opt/vela/licenses.db "INSERT INTO licenses ..." (paste the SQL)`

RUN `sqlite3 /opt/vela/licenses.db "SELECT client_id, status FROM licenses;"`

✓ GOOD

`ricardo_rossello|active|command`

✗ PROBLEM

Empty result or wrong status

3 Test validation

From your Mac Mini — not the droplet.

RUN `curl -s -X POST http://165.22.36.184:8080/validate -H "Content-Type: application/json" -d '{"license_key": "VELA-XXXX-XXXX-XXXX-XXXX"}'`

✓ GOOD

`{"status": "active", "tier": "command", ...}`

✗ PROBLEM

`{"status": "invalid", ...}`

4 Send the client their pre-install materials



REMOTE FACILITATOR GUIDE

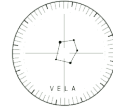
- README First PDF — they need 45–90 min to collect credentials
- Their VELA License Key: VELA-XXXX-XXXX-XXXX-XXXX
- The install call time — block 90 minutes

5 Confirm they have done the pre-install checklist

Ask them directly:

- Anthropic account created and billing configured?
- Telegram bot created — token in hand?
- Telegram User ID and Group ID collected?
- Google Cloud project created — OAuth credentials downloaded?
- Gmail API, Calendar API, Drive API all enabled?
- VELA Backups folder created in Google Drive?
- Mac Mini Ethernet connected? Energy Saver set to Never?

Do not start the install call if the client has not completed the pre-install checklist. Incomplete credentials will stop the installation at a later step and waste the call.



02 Opening the Install Call

First 5 minutes.

1 Get on a video call

Zoom, FaceTime, or whatever they prefer. You need to see their screen.

2 Ask them to share their screen

You need to see Terminal clearly.

3 Confirm prerequisites visually

- Ethernet cable visible at the back of the Mac Mini
- Energy Saver already set to Never (check together if needed)
- All credentials open in their password manager

4 Set up your Tailscale SSH session

Before they run anything. You need your own Terminal window watching.

Ask the client for their Mac Mini username. Ask them to open System Preferences → Sharing and confirm Remote Login (SSH) is ON.

RUN (your Mac) `ssh [username]@[tailscale-ip-or-hostname]`

FACILITATOR NOTE

You won't have the Tailscale IP yet until after Tailscale is installed. That happens in Step 3 of monitoring setup post-install. For now, if they have Tailscale already installed, use it. If not, you'll SSH in after monitoring setup is complete.

03 Watching the Install — Section by Section

Section 1 — System Check

They paste and run the one-liner. You watch their screen.

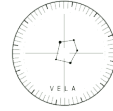
THEY RUN `curl -fsSL https://raw.githubusercontent.com/greg442/Vela_Scripts_2/main/install.sh | bash`

✓ GOOD

- ✓ macOS 15.x — Apple Silicon
- ✓ 32 GB unified memory
- ✓ Internet confirmed

✗ PROBLEM

- ✗ 16 GB RAM minimum required
- ✗ No internet connection

**FACILITATOR NOTE**

If RAM fails: installation cannot continue on this hardware. The minimum is 16 GB. If internet fails: check Ethernet physically, check router. Try: ping 8.8.8.8 in a separate Terminal tab.

If sleep warning appears, walk them through:

THEY DO

Apple menu → System Settings → Energy → "Prevent automatic sleep" → ON → return to Terminal → press Enter

Section 2 — License Validation

The installer prompts for the license key. Read it to them or have it in their password manager.

✓ GOOD

✓ License valid — Tier: command

✗ PROBLEM

✗ Invalid license key

FACILITATOR NOTE

If invalid: confirm they typed it correctly — no extra spaces, correct format VELA-XXXX-XXXX-XXXX-XXXX. If still failing, SSH into the droplet and confirm the key_hash is in the database: `sqlite3 /opt/vela/licenses.db "SELECT * FROM licenses WHERE client_id = 'CLIENT_ID';"`

Section 3 — Client Information

Walk through each prompt with them. Key things to watch:

- Mac Mini username — must match their actual login. Ask them to open Terminal and type: `whoami` to confirm.
- Timezone — must be exact format: America/New_York, America/Los_Angeles, America/Chicago, America/Denver, Europe/London
- Telegram Group ID — must be negative integer. If they got a positive number, they got their user ID twice. Go back to @userinfobot inside the group.
- Anthropic key — must start with sk-ant-. If it doesn't, they're using the wrong key.

✓ GOOD

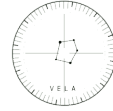
Continue? (y/n): y
(all details look correct)

✗ PROBLEM

Wrong email / wrong group ID
/ positive group ID

FACILITATOR NOTE

If they make a mistake at the confirm step: type n, the installer exits. They restart with: up arrow + Enter to re-run the same command. All prompts repeat.



Section 4 — Downloading Scripts

✓ GOOD	✗ PROBLEM
Downloaded 12 scripts	More than 2 download warnings

FACILITATOR NOTE

If downloads fail: the GitHub repo may be private or the raw URL is blocked. Check: `curl -I https://raw.githubusercontent.com/greg442/Vela_Scripts_2/main/install.sh` from their Terminal. If 404, push the files to GitHub first.

Step 3 — Ollama Model Download (15–30 min)

This is the long step. Tell the client: 'This is downloading the AI models. It normally takes 15–30 minutes. You will see a progress bar. Just leave it running.'

✓ GOOD	✗ PROBLEM
pulling manifest	Error: connection refused
pulling [#####...] 4.5 GB	or error pulling model

FACILITATOR NOTE

If model download fails: check internet speed. Try manually: `ollama pull qwen2.5:7b` in a new Terminal tab. If Ollama server isn't running: `ollama serve` & then retry.

Step 9 — Google Authentication

A browser opens automatically. Walk them through it.

1 Browser opens to Google sign-in

THEY DO Click their primary Gmail account → Allow

2 Installer asks about CoS email

THEY TYPE y + Enter

THEY DO Sign into CoS Gmail → Allow

FACILITATOR NOTE

If browser doesn't open: the installer shows a URL in Terminal. Have them copy-paste it into Chrome. If Google shows 'This app isn't verified': click Advanced → Go to VELA (unsafe). This is expected for a new Google Cloud project that hasn't been verified yet.

Step 18 — Start and Verify



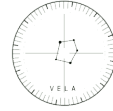
REMOTE FACILITATOR GUIDE

Watch for the workspace map output. This is the most important verification step.

✓ GOOD	✗ PROBLEM
<pre>cos anthropic/claude- sonnet-4-6 /Users/.../workspace-cos</pre>	<pre>cos anthropic/claude- sonnet-4-6 (none)</pre>
<pre>analyst anthropic/claude- sonnet-4-6 /Users/.../workspace-analyst</pre>	<pre>or any agent showing (none)</pre>

FACILITATOR NOTE

If any agent shows (none): SSH into their machine and run the Python workspace fix manually. See Section 05 of this guide.



04 Post-Install Steps — You Run These

Set Up Monitoring

THEIR TERMINAL `bash ~/.openclaw/scripts/monitoring/setup_monitoring.sh`

When prompted, enter:

Prompt	Enter
Client name	Their full name (e.g. Ricardo Rossello)
VELA Monitor bot token	8632086745:AAF5Nhp-WbGQHOCwJgDMP1khSSKqMyLFwa0
VELA Monitor chat ID	8248659673
Tier	command

You receive a Telegram confirmation from @VELAMonitor_bot. That's the confirmation it worked.

Set Up Tailscale

THEIR TERMINAL `bash ~/.openclaw/scripts/monitoring/setup_tailscale.sh`

Provide the Tailscale auth key when prompted. Generate one at tailscale.com/admin → Auth Keys → Generate key.

After setup, the machine appears in your Tailscale dashboard as vela-[client_id]. Test your SSH access:

YOUR TERMINAL `ssh [username]@vela-[client_id]`

✓ GOOD

Connected to client machine

✗ PROBLEM

ssh: Could not resolve hostname

Pair Telegram

Ask the client to open Telegram on their phone and message their bot. Walk them through:

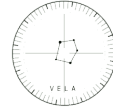
1 Search for their bot
Search for the username they created with @BotFather

2 Send a message
Type: Hello and send

3 Confirm Hannah responds
She should respond within 10 seconds

✓ GOOD

✗ PROBLEM



REMOTE FACILITATOR GUIDE

Hannah responds in Telegram

No response after 30 seconds

FACILITATOR NOTE

If no response: SSH into their machine and run: `openclaw gateway status`. If gateway is down: `openclaw gateway restart`. Check logs: `tail -50 ~/.openclaw/logs/gateway.err.log`

Pair WhatsApp

Ask the client to:

THEY DO

WhatsApp → Settings → Linked Devices → Link a Device → scan QR on screen

FACILITATOR NOTE

The QR code appears in the Mac Mini terminal window or OpenClaw interface. If it's not visible, run: `openclaw gateway restart` and look for the QR prompt.

Test Email Triage**SSH FROM YOUR MACHINE**

`python3 ~/.openclaw/scripts/email_triage.py`

✓ GOOD

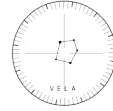
Triage complete. N emails processed.

✗ PROBLEM

`gog-wrapper auth error`

FACILITATOR NOTE

If auth error: run `gog-wrapper auth login -a CLIENT_EMAIL` on their machine. Browser will open for re-auth.

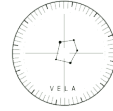


05 Troubleshooting Reference

Problems that require your direct intervention.

Agent Shows (none) Workspace

Run the Python workspace fix directly on their machine via SSH:



REMOTE FACILITATOR GUIDE

```
# SSH into their machine first
ssh [username]@vela-[client_id]

# Run the workspace fix
cp ~/.openclaw/openclaw.json ~/.openclaw/openclaw.json.bak.$(date +%Y%m%d%H%M)

python3 << 'EOF'
import json, os
path = os.path.expanduser('~/.openclaw/openclaw.json')
username = os.path.expanduser('~').split('/')[1]
base = f'/Users/{username}/.openclaw'
with open(path) as f:
    c = json.load(f)
workspace_map = {
    'cos': f'{base}/workspace-cos',
    'analyst': f'{base}/workspace-analyst',
    'marketing': f'{base}/workspace-marketing',
    'pm': f'{base}/workspace-pm',
    'researcher': f'{base}/workspace-researcher',
    'legal': f'{base}/workspace-legal',
}
for a in c.get('agents', {}).get('list', []):
    if a.get('id') in workspace_map:
        a['workspace'] = workspace_map[a['id']]
        print(f"Set {a['id']} -> {workspace_map[a['id']]}")
with open(path, 'w') as f:
    json.dump(c, f, indent=2)
print('Done.')
EOF

openclaw gateway restart
```



Google Auth Error on Email Triage

```
# On their machine via SSH

gog-wrapper auth login -a CLIENT_PRIMARY_EMAIL

gog-wrapper auth login -a CLIENT_COS_EMAIL


# Verify both accounts connected

gog-wrapper gmail search "is:unread" -a CLIENT_PRIMARY_EMAIL --limit 3
```

Gateway Won't Start

```
# Check what's blocking it

openclaw gateway status

lsof -i :3000


# Full reinstall of gateway

openclaw gateway stop

openclaw gateway install

openclaw gateway start


# Check logs for error

tail -50 ~/.openclaw/logs/gateway.err.log
```

Ollama Models Missing

```
# Check what's installed

ollama list


# Pull missing models manually

ollama serve &

ollama pull qwen2.5:7b

ollama pull llama3.1:8b
```



cacheRetention Wrong

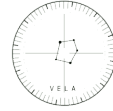
```
# Verify — should say 'short'

python3 -m json.tool ~/.openclaw/openclaw.json | grep cacheRetention


# Fix if wrong

openclaw config set agents.defaults.models."anthropic/claude-sonnet-4-6".params.cacheRetention
short

openclaw gateway restart
```



06 Final Verification Checklist

Run through this before ending the call.

System

- ☐ openclaw status — all green, no red errors
- ☐ openclaw agents — all 6 agents listed, no (none) workspace entries
- ☐ openclaw cron list — hannah-morning and hannah-evening present
- ☐ ollama list — qwen2.5:7b and llama3.1:8b present
- ☐ openclaw.json.bak.stable exists

Channels

- ☐ Hannah responds to a Telegram message from the client
- ☐ WhatsApp QR scanned and confirmed
- ☐ gog-wrapper shows emails from client's Gmail
- ☐ email_triage.py runs without error

Monitoring

- ☐ @VELAMonitor_bot sent a Telegram confirmation during setup
- ☐ Uptime Kuma running — open <http://localhost:3001> on their machine via Tailscale
- ☐ backup_gdrive.sh --setup completed — test backup confirmed in Drive
- ☐ Client machine appears in your Tailscale dashboard as vela-[client_id]
- ☐ SSH access confirmed from your machine

Cost & License

- ☐ python3 ~/.openclaw/scripts/cost_alert.py runs without error
- ☐ cat ~/.openclaw/.vela_tier shows active tier
- ☐ No .vela_standdown file present

07 Post-Install — Update Your Records

1 Update the client manifest

YOUR TERMINAL `nano ~/.vela/deployments/[client_id].conf`

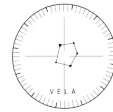
Fill in: TAILSCALE_IP, INSTALL_DATE, ANTHROPIC_ACCOUNT, TELEGRAM_BOT, TELEGRAM_GROUP_ID

2 Schedule Day 3 check-in

Send a personal message 3 days after install: 'How's Hannah treating you? Anything to calibrate?'

3 Schedule Day 7 review

Check API cost trend in cost-alert.log. Ask if brief timing or format needs adjustment. Update HEARTBEAT.md or DISPATCH_RULES.md if needed.



08 Key Credentials — Keep This Secure

Item	Value
License server IP	165.22.36.184
License server SSH	ssh root@165.22.36.184
Admin key	vela-admin-d2724b0584e53dfe7dd35c37393f484
VELA Monitor bot token	8632086745:AAF5Nhp-WbGQHOCwJgDMP1khSSKqMyLFwa0
VELA Monitor chat ID	8248659673
Client manifests	~/.vela/deployments/
Tailscale dashboard	tailscale.com/admin/machines
Admin CLI	python3 ~/Downloads/vela-scripts\ 2/license_server/admin.py list

This document contains sensitive credentials. Store in 1Password. Do not email, screenshot with values visible, or share with clients.